IILM UNIVERSITY
Greater Noida

PYTHON INTERNSHIP PROJECT

Project Portfolio, Automation & Web Deployment

Submitted By

Ashwina Pandey

Roll No.: 25SCS1003004835

B.Tech. CSE • Section 2CSE4

IILM University

Academic Year: 2025–2029

Project Overview

Python Sandbox

Isolated Python environment, entry-point script and verification logging.

Directory Cleanup

Automated organization of documents and images by file extension.

Image Downloader

Reads image URLs and downloads multiple images using requests.

Cron Scheduler & Logger

Simulated scheduled executions with timestamps and execution statistics.

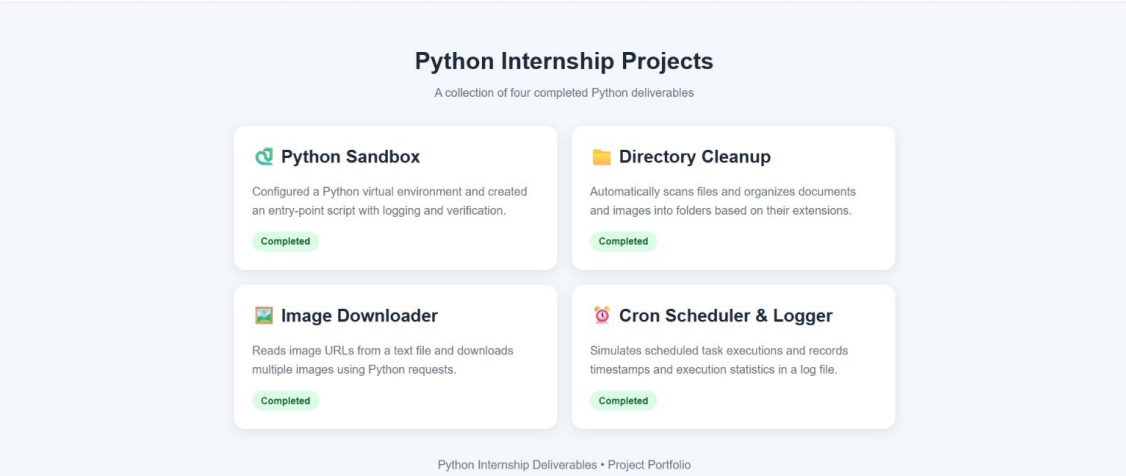


Figure 2.1: Project portfolio web interface showing all four completed deliverables.

Objectives

- Build practical Python programs using core libraries and modules.
- Understand virtual environments, dependencies and project structure.
- Automate file organization and image downloading tasks.
- Implement task scheduling concepts and persistent execution logging.
- Develop a Flask-based web interface for the completed deliverables.
- Use Git and GitHub for version control and project hosting.
- Deploy the web application to a public cloud platform using Render.
- Practice testing, debugging, documentation and software delivery.

Technologies Used

Python

Core programming language

Flask

Web application framework

Requests

HTTP requests & image downloads

OS / Shutil

File and directory operations

Logging / Time

Execution records & timing

Git / GitHub

Version control & repository

Gunicorn

Production WSGI server

Render

Cloud deployment

Task 1 — Python Sandbox Setup

- Created an isolated Python virtual environment for the project.
- Used logging to verify that the environment and entry-point script were working.
- Maintained dependencies through requirements.txt.
- Verified execution successfully from the terminal.

main.py

```python
import logging

logging.basicConfig(level=logging.DEBUG)

logging.debug("Python sandbox is running!")
logging.info("Python environment verified successfully.")
```

Figure 5.1: main.py entry-point and logging configuration.

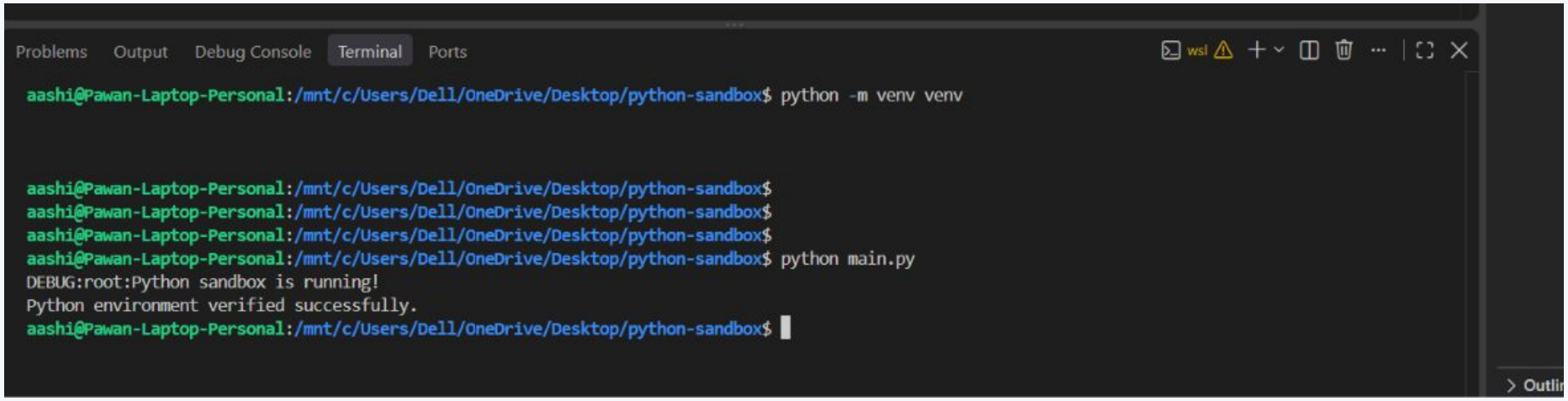


Figure 5.2: Successful Python Sandbox verification in the terminal.

Task 2 — Directory Cleanup

- Scanned a target directory and identified files by extension.
- Mapped document and image extensions to destination folders.
- Used os and shutil for directory creation and file movement.
- Successfully separated documents and images into organized folders.

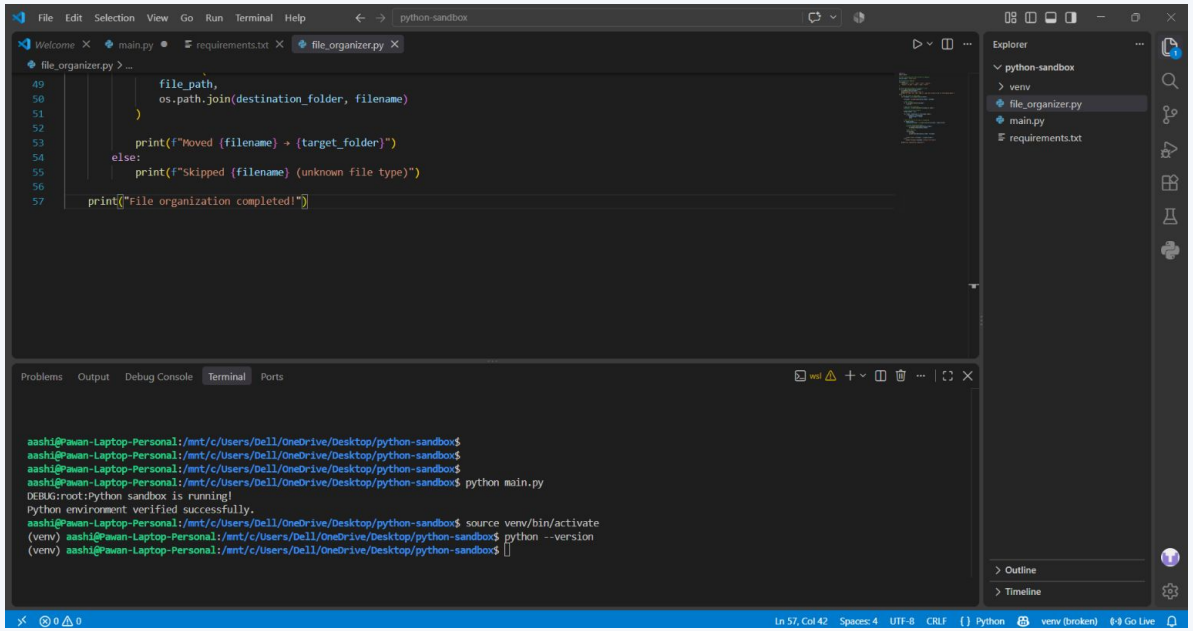


Figure 6.1: file_organizer.py implementation.

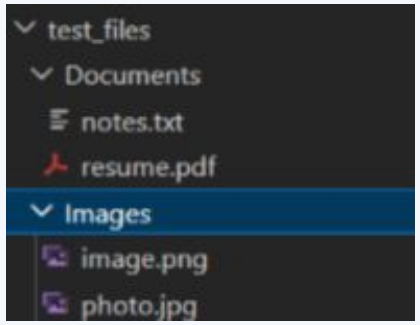


Figure 6.2: Organized Documents and Images folders.

Automation Flow

Scan files → Detect extension → Create destination → Move file

Task 3 — Image Downloader

- Stored image URLs in image_urls.txt.
- Used requests to retrieve image content over HTTP.
- Used ThreadPoolExecutor to download multiple images efficiently.
- Saved successfully downloaded images in the downloaded_images folder.
- Handled network and file-related exceptions.

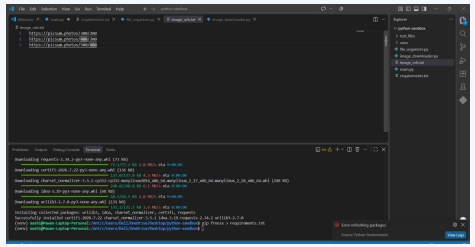


Figure 7.1: Input image URLs.

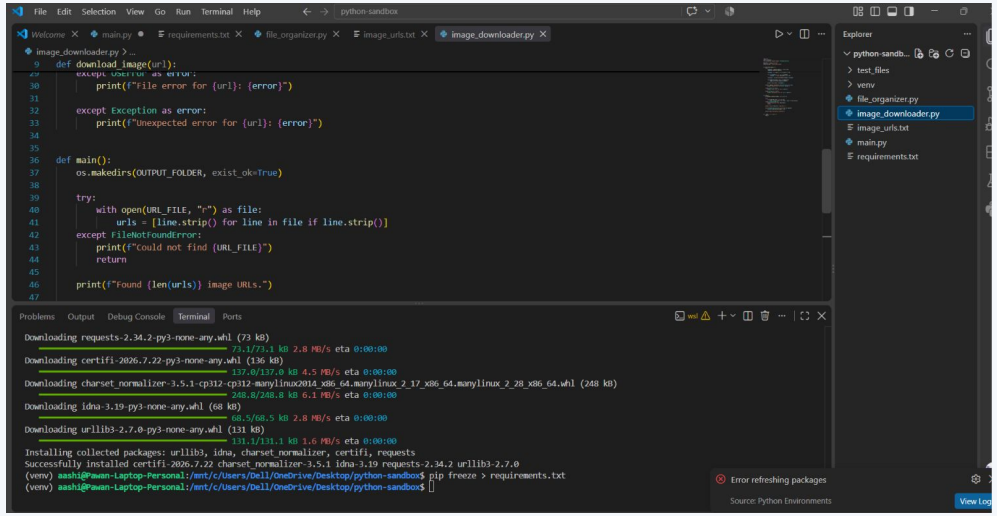
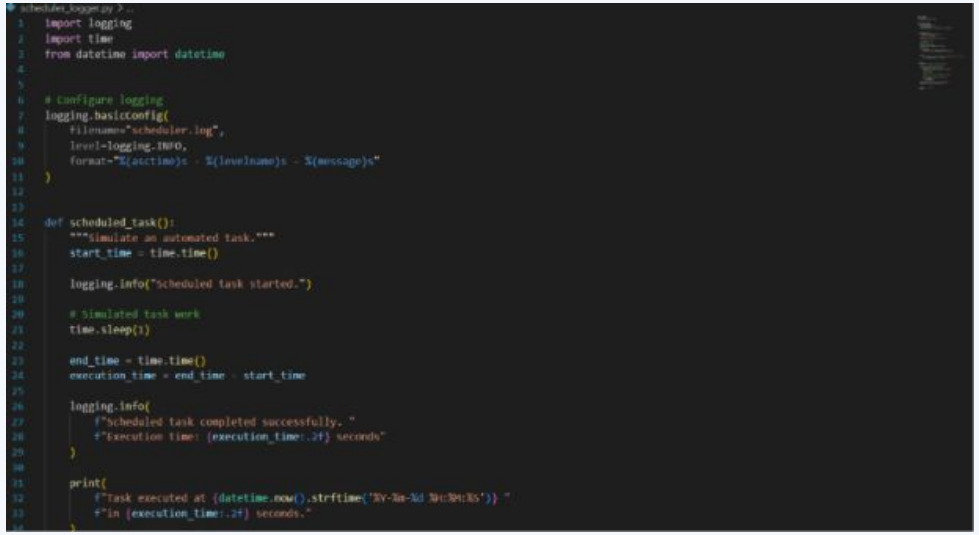


Figure 7.2: Image downloader implementation.

Figure 7.3: Successful image download output.

Task 4 — Cron Scheduler & Logger

- Simulated three scheduled task runs.
- Recorded run numbers, timestamps and task status.
- Measured execution time for each scheduled task.
- Stored execution history in scheduler.log.
- Confirmed all scheduled runs completed successfully.



```python
import logging
import time
from datetime import datetime


# Configure logging
logging.basicConfig(
    filename="scheduler.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)


def scheduled_task():
    """Simulate an automated task."""
    start_time = time.time()

    logging.info("Scheduled task started.")

    # Simulated task work
    time.sleep(1)

    end_time = time.time()
    execution_time = end_time - start_time

    logging.info(
        f"Scheduled task completed successfully. "
        f"Execution time: {execution_time:.2f} seconds"
    )

    print(
        f"Task executed at {datetime.now().strftime('%Y-%m-%d %H:%M:%S')} "
        f"in {execution_time:.2f} seconds."
    )
```

Figure 8.1: Scheduler and logging implementation.

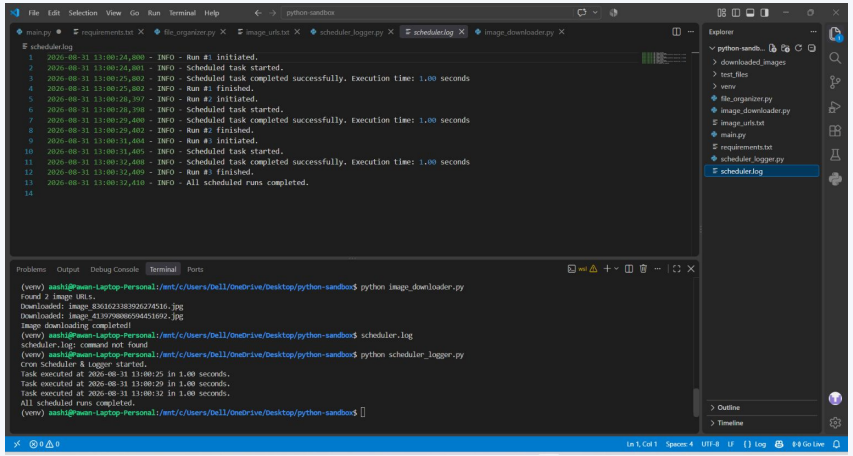


Figure 8.2: Scheduler log and terminal output showing three successful runs.

Web Application

- Developed a Flask-based project portfolio interface.
- Displayed all four completed Python deliverables.
- Styled the interface using HTML and CSS.
- Tested the application locally before deployment.
- Configured the application to run on the required host and port.

Figure 9.1: Flask application code.

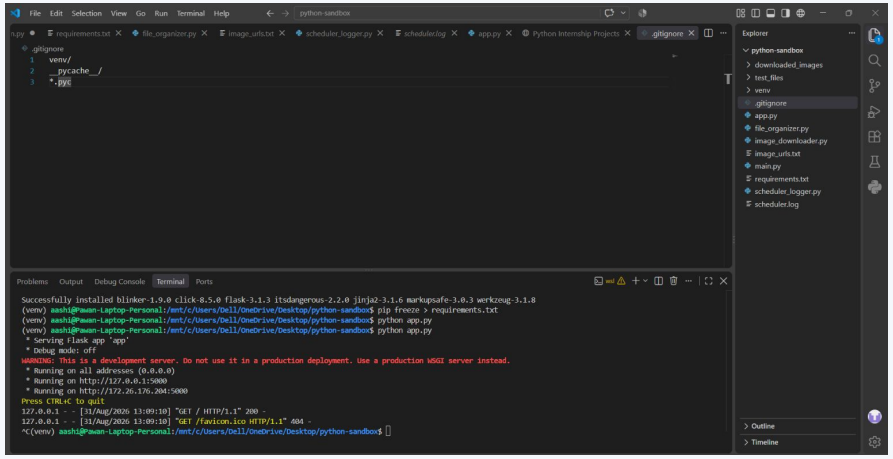


Figure 9.2: Local Flask application running successfully.

GitHub Version Control

- Initialized and maintained the project using Git.
- Committed project changes with meaningful commit messages.
- Pushed the main branch to GitHub.
- Repository is public and contains the completed source files.
- GitHub provides a clear version history and project structure.

Repository
github.com/ashwina0202-ai/python-internship-project

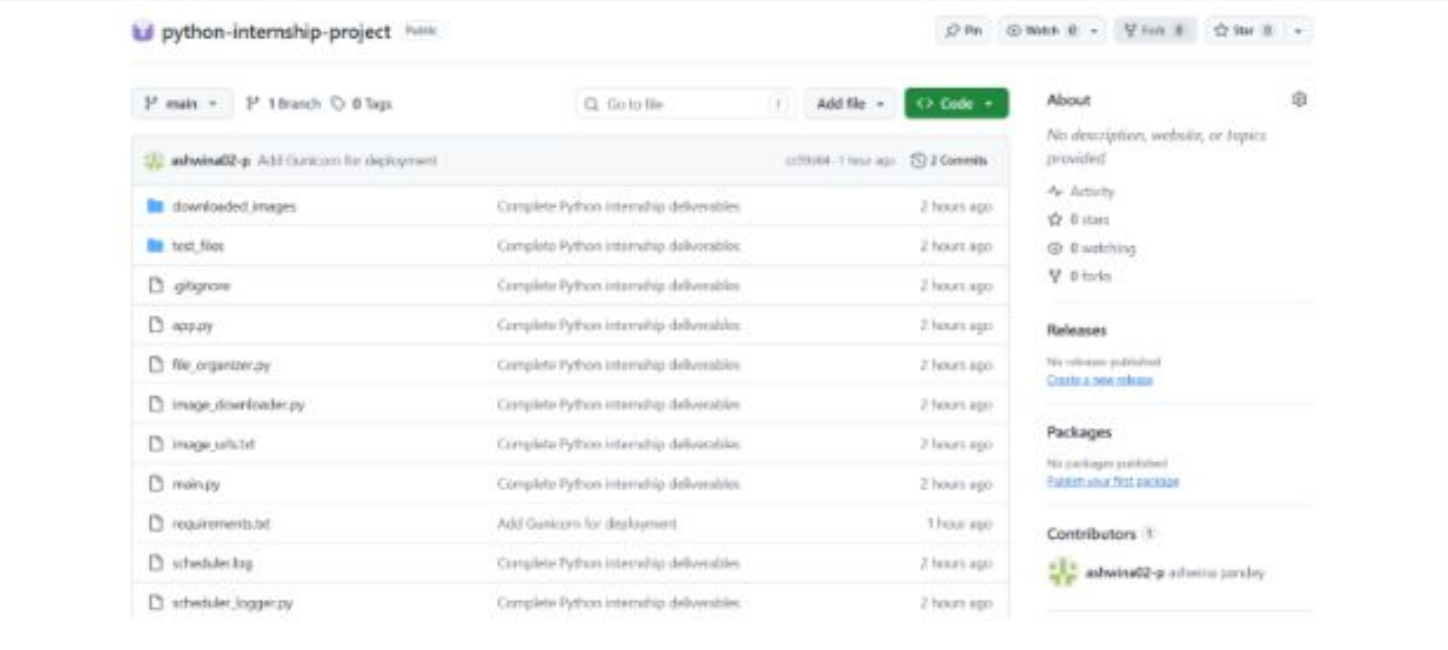


Figure 10.1: Public GitHub repository containing the internship project.

Live Deployment

- Configured Gunicorn as the production WSGI server.
- Connected the Render service to the GitHub repository.
- Deployed the Flask application as a public web service.
- Verified the live application after deployment.

Live Application
python-internship-project-m0ny.onrender.com

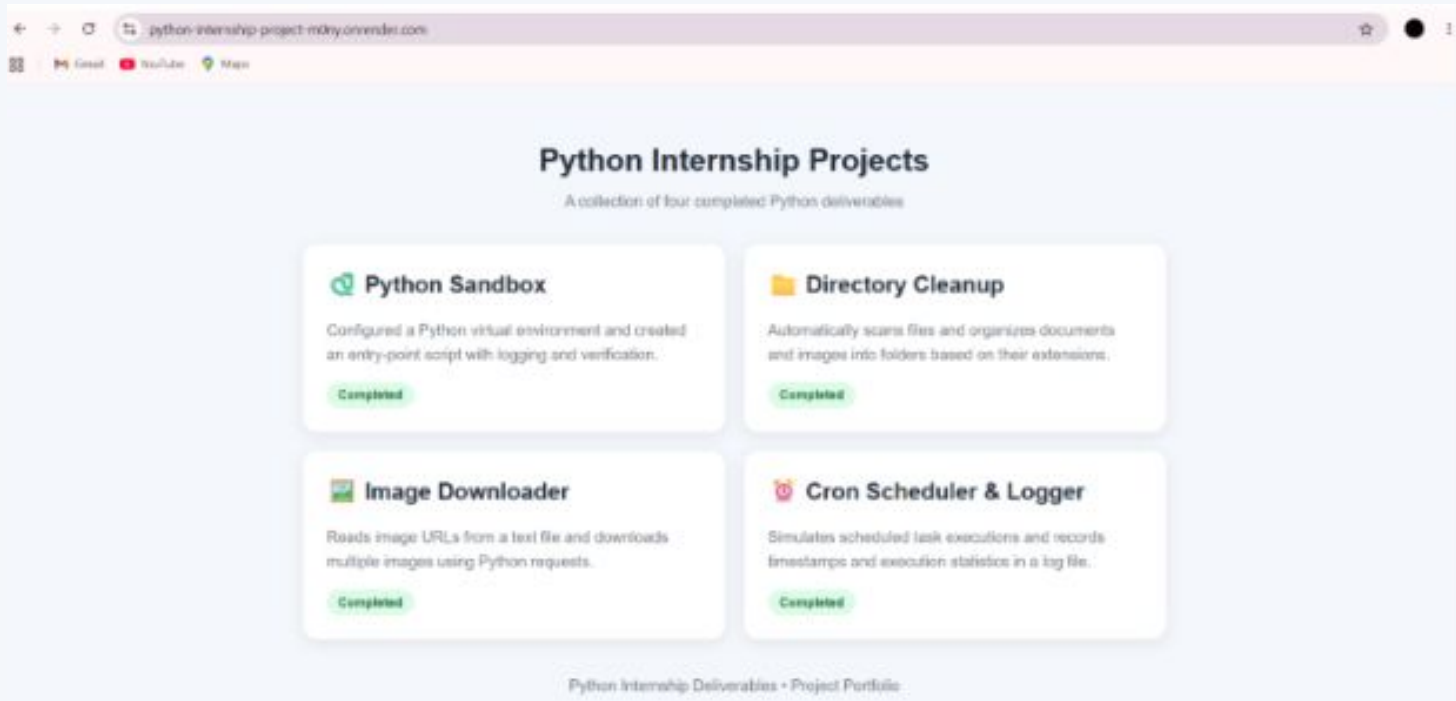


Figure 11.1: Live Python Internship Projects application deployed on Render.

Testing & Results

Python Sandbox

Environment verified successfully

Directory Cleanup

Files organized by extension

Image Downloader

Multiple images downloaded successfully

Scheduler & Logger

Three scheduled runs completed and logged

Flask Web App

Local application tested successfully

GitHub

Source code pushed to public repository

Render

Live application deployed successfully

Conclusion & Future Scope

Conclusion

The internship project provided practical experience across Python programming, automation, file handling, HTTP requests, logging, Flask web development, Git/GitHub, testing and cloud deployment.

All four core deliverables were implemented, tested and presented through a live web application.

Future Scope

- Add a database for execution history
- Add authentication and interactive controls
- Add real-time monitoring and notifications
- Expand file-type support and testing
- Improve analytics and reporting

INTERNSHIP CERTIFICATE

THANK YOU

Python Internship Project

Ashwina Pandey

25SCS1003004835 • B.Tech. CSE • 2CSE4

IILM University

GitHub: github.com/ashwina0202-ai/python-internship-project

Live: python-internship-project-m0ny.onrender.com